

Highlights

A Visual Low-Code Robotic Control Framework with Hybrid Semantic-Geometric Perception for Non-Expert Operators

Xiaoqi Zhou, Lianqing Zhong, Zixuan Kuang, Huan Yang

- Visual low-code framework lowers robot programming barrier for non-expert workers.
- Integrates VLM and YOLOv8 for precise hybrid semantic-geometric perception.
- Pre-execution safety checker prevents tabletop collisions and out-of-bound errors.
- User study shows 72.6% time reduction and highly improved system usability (SUS 77.2).

A Visual Low-Code Robotic Control Framework with Hybrid Semantic-Geometric Perception for Non-Expert Operators

Xiaoqi Zhou^{a,1}, Lianqing Zhong^{a,1}, Zixuan Kuang^a and Huan Yang^{a,*}

^a*Aberdeen Institute of Data Science and Artificial Intelligence, South China Normal University, Nanhai Campus, Shishan Town, Nanhai District, Foshan, 528225, Guangdong, China*

ARTICLE INFO

Keywords:

Low-Code
Robotic Arm Control
Human-Robot Interaction
Hybrid Perception
Visual Programming
Industrial Automation

ABSTRACT

With the rapid growth of industrial automation, there is a critical shortage of skilled robotics engineers, leaving a significant "skill gap" between front-line operators who possess valuable domain process knowledge and the complex direct programming paradigms required to operate modern industrial robots. To address this challenge, this paper presents a web-based, visual, low-code robotic arm control framework designed to lower the barrier to entry for non-expert vocational workers. Leveraging a graphical block-based syntax, the proposed framework reconstructs the robot programming workflow into an intuitive human-computer mapping pipeline. Technically, the framework integrates a hybrid semantic-geometric perception engine and an active safety validation mechanism. It combines a lightweight Vision-Language Model (VLM) for topological scene graph generation with a geometric object detector (YOLOv8) to resolve spatial precision issues, transforming 2D pixel coordinates into precise 3D physical coordinates via camera calibration. Furthermore, a pre-execution safety checker statically validates generated paths prior to physical movement, preventing out-of-boundary actions and tabletop collisions. We validated the framework through real-world sorting tasks on a 3-axis industrial robotic arm with an "Eye-in-Hand" configuration, and conducted a controlled user study with eight participants. The quantitative results demonstrate a profound efficiency and safety advantage over conventional direct programming: our low-code framework reduces average task completion time by up to 72.6%, maintains a low average error rate of less than 1.0 per task, and substantially enhances user experience, achieving a System Usability Scale (SUS) score of 77.2 compared to the direct programming baseline's 50.6. This work demonstrates the high potential of low-code, perception-enabled interfaces in flattening the learning curve for advanced industrial automation.

1. Introduction

1.1. Background

The global manufacturing industry is currently experiencing a rapid surge in the demand for automation; however, there is a severe shortage of professional robotics engineers equipped with advanced programming capabilities. According to the Future of Jobs Report 2025 by the World Economic Forum, 58% of global enterprises plan to accelerate the adoption of robotics and autonomous systems, yet 63% of employers cite "skill gaps in the labor market" as the primary barrier to this transformation (World Economic Forum). This highlights a core contradiction in modern factories: on one hand, a large number of vocationally trained workers possess invaluable domain and process knowledge; on the other hand, they lack the complex coding skills required to operate modern industrial robots (e.g., C++, Python, or ROS). As illustrated in Fig. 1 (Panel A), when front-line workers familiar with traditional Ladder Logic are forced to confront text-based programming interfaces, they face a steep learning curve and frequent execution failures due to syntax or logic errors (Kumar, Jetley and Sureka, 2016; Kamel and Kamel, 2019). Consequently, the high barrier to entry of existing robot development platforms severely hinders this segment of the workforce from maximizing their value, creating an urgent need for a low-barrier tool that seamlessly bridges "process knowledge" with "machine instructions".

*Corresponding author

✉ 20233802036@m.scnu.edu.cn (X. Zhou); 20233803064@m.scnu.edu.cn (L. Zhong); 20223801046@m.scnu.edu.cn (Z. Kuang); huan.yang@m.scnu.edu.cn (H. Yang)

ORCID(s):

¹These authors contributed equally to this work.

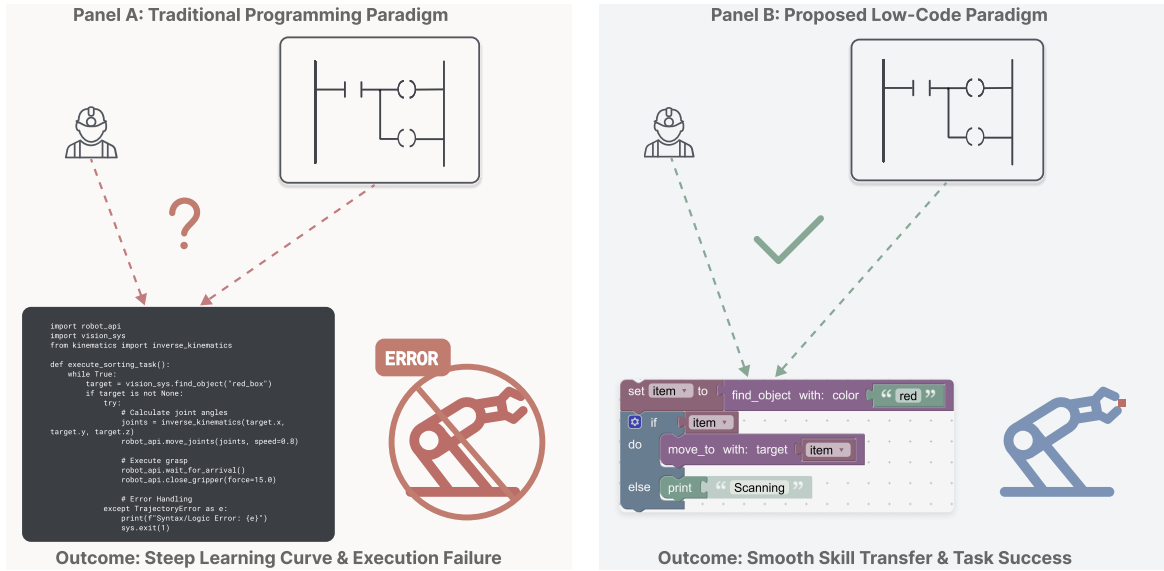


Figure 1: Comparison between the traditional robot programming approach and the proposed low-code framework. (a) Traditional text-based programming imposes a steep learning curve and high error rates on field workers. (b) The proposed low-code environment bridges the skill gap, allowing workers to intuitively map domain knowledge into control logic.

1.2. Motivation: Why Low-Code

To lower the programming barrier, traditional teach pendants and Programming by Demonstration (PbD) are widely adopted in the industry. However, these methods are often limited to recording fixed spatial trajectories and struggle to accommodate complex control flows, such as conditional branching and iterative operations (Fischer, Kirstein, Jensen, Krüger, Kukliński, aus der Wieschen and Savarimuthu, 2016). To bridge the gap between intuitive operation and complex logic expression, we introduce a visual low-code programming framework based on Blockly.

Reason 1: The imperative need for fault tolerance in vocational training. The primary objective of our system is to facilitate skill training and engineering education for front-line workers. In the modern robotics development lifecycle, Virtual Prototyping serves as a critical phase connecting conceptual design with physical deployment. Recent studies indicate that digital twin and virtual simulation environments effectively mitigate the risk of hardware collision and alleviate the cognitive load associated with physical safety constraints (Erdei, Krakó and Husi, 2022). The proposed low-code framework provides a fault-tolerant sandbox, empowering operators to rapidly iterate control logic. By abstracting away strict syntactical requirements, the system enables workers to focus entirely on task-level process orchestration rather than low-level code debugging.

Reason 2: The cognitive and structural similarity between Blockly and Ladder Logic. For decades, the programming of industrial Programmable Logic Controllers (PLCs) has been predominantly guided by the IEC 61131-3 standard, which established Ladder Logic as its cornerstone graphical language. Fundamentally, Ladder Logic constitutes a graphical logic representation constructed through the spatial arrangement of functional modules. This paradigm shares profound structural and interactive similarities with the block-based semantics of Blockly. While front-line operators may lack formal training in text-based programming, their vocational education and daily operational routines are deeply rooted in Ladder Logic methodologies. Consequently, the introduction of a Blockly-based interface does not impose the burden of acquiring an entirely new programming skill set; instead, it effectively recontextualizes their pre-existing understanding of industrial logic. This seamless skill transfer renders Blockly an optimal bridge, connecting the operators' established cognitive frameworks with advanced robotic control mechanisms.

1.3. Contributions

To address these challenges, this paper presents the design and development of a web-based, low-code robotic arm control framework, specifically targeting the critical need to expedite the training of existing skilled field workers and

lower the technical barrier for comprehending sophisticated engineering principles. The major contributions of this paper are as follows:

- (1) We propose a visual, low-code control architecture tailored for non-expert users, seamlessly integrating a frontend interactive UI with backend execution.
- (2) We developed an automated pipeline to translate graphical blocks into robot commands by using a hybrid perception mechanism. This pipeline integrates a lightweight VLM and object detection for precise semantic to spatial mapping while supporting complex logic control.
- (3) Through real-world scenario deployment, we validate the effectiveness of the system from two dimensions: technical feasibility and system usability, demonstrating its potential in lowering the operational barriers of industrial robots.

2. Related Work

2.1. Visual Programming for Industrial Applications

Visual programming interfaces, ranging from standardized Ladder Logic (Sehr, Lohstroh, Weber, Ugalde, Witte, Neidig, Hoeme, Niknami and Lee, 2021) to data-flow tools like LabVIEW (Kodosky, 2020) and block-based environments such as Scratch or Blockly (Weintrop, 2019), have long been established as essential catalysts for industrial automation. Unlike conventional direct programming, these visual paradigms leverage "visual logic representation" to correspond with the intuitive reasoning of non-experts. Empirical studies by Kumar et al. (2016) and Weintrop (2019) confirm that such interfaces significantly reduce cognitive load by providing visual cues and utilizing the principle of "syntax error mitigation" to prevent coding errors. Furthermore, Mayr-Dorn, Winterer, Salomon, Hohensinger and Ramler (2021) demonstrated that these tools possess sufficient expressive power to implement highly complex industrial robot programs. By shielding users from low-level intricacies, visual interfaces enable vocational workers to successfully deploy automated tasks that conform to their existing operational experience.

2.2. Low-Code Development

In the robotics domain, low-code paradigms have evolved from simple motion recording to structured logic composition. While Programming by Demonstration (PbD) offers an intuitive way to teach robots, Fischer et al. (2016) noted that it often lacks the flexibility to handle conditional branching and iterative logic. To address this, Robot Blockly (Weintrop, Shepherd, Francis and Franklin, 2017) was introduced, proving that adult novices could successfully author functional pick-and-place routines. However, a critical limitation persists: these systems remain "blind" to their environment. As highlighted by Steinmetz, Nitsch and Stulp (2019), even advanced task-level systems frequently require manual coordinate teaching due to a lack of autonomous scene recognition. While recent generative AI research (Wang, Shi, Hu, Ma, Liu, Wang, Yao, Liu, Ge and Zhang, 2025; Zeng, Gan, Huai, Sun, Chen, Wang, Liu and Yu, 2025) suggests using VLMs for end-to-end planning, these "black-box" approaches often bypass the structured safety constraints essential for industrial environments. Consequently, there remains a gap for a framework that combines the logical rigor of Blockly with the hybrid perception of lightweight VLMs and precise object detection—a challenge this paper aims to solve.

3. Problem Statement

To formalize the challenges faced by front-line workers and the rationale behind our system design, we model the robotic programming workflow using mathematical abstractions.

3.1. Formalization of the Skill Gap

Let Task T denote the industrial operation that needs to be executed by the robot. The successful programming of T relies on two primary input variables: K_{proc} , which represents the worker's domain process knowledge (e.g., "pick up the red box and place it on the conveyor"), and S_{code} , which denotes the worker's programming skills.

In traditional control paradigms, generating executable code requires a direct mapping function $f(K_{\text{proc}}, S_{\text{code}}) \rightarrow C_{\text{exec}}$, where C_{exec} is the Executable Code. However, for typical vocational workers, $S_{\text{code}} \approx \emptyset$. As highlighted by Kumar et al. (2016), industrial logic programming heavily relies on specialized structural syntax, making traditional text-based programming excessively complex for non-experts to comprehend and debug. Consequently, the function f cannot be manually completed by front-line operators.

Proposed Solution: To bypass the lack of coding skills, we introduce a visual intermediate representation, denoted as V_{block} . We reconstruct the infeasible direct mapping f into a composition of two sequential mapping functions, g and h , such that:

$$C_{\text{exec}} = h(g(K_{\text{proc}})) \quad (1)$$

Specifically, $g : K_{\text{proc}} \rightarrow V_{\text{block}}$ represents the Human-Computer Interaction (HCI) mapping. Here, the worker's process intention is intuitively transformed into graphical blocks, relying entirely on K_{proc} without requiring S_{code} . Subsequently, $h : V_{\text{block}} \rightarrow C_{\text{exec}}$ represents the automated compilation mapping, where the system translates the assembled block structures into executable robot commands.

3.2. Abstraction of Intelligent Control Workflow

We further model the system workflow as a closed-loop control abstraction consisting of semantic understanding, geometric localization, spatial mapping, and safety control.

1. Hybrid Perception & Spatial Mapping:

- Semantic Scene Understanding:** We define the function $M_{\text{vlm}}(I) \rightarrow G_{\text{sem}}$, where the Vision-Language Model (VLM) processes the raw input image I to output a semantic scene graph G_{sem} . This graph contains object labels and their relative topological relations (e.g., "red cube is left of blue cube").
- Geometric Localization:** We define the target detection function $M_{\text{det}}(I, O_{\text{target}}) \rightarrow (u, v)$. Given the image I and a target object label O_{target} (e.g., "red"), the object detector (YOLOv8) outputs the precise pixel centroid coordinates (u, v) .
- Spatial Transformation:** We define the mapping function $M_{\text{geo}}(u, v) \rightarrow P_{\text{world}}$, which transforms pixel coordinates (u, v) to physical 3D coordinates $P_{\text{world}} = (x, y, z)$ using a calibrated camera affine transformation matrix.

2. Logic Synthesis & Safe Execution:

- Rule-Based Action Generation:** We define the control logic function $M_{\text{ctrl}}(S) \rightarrow A$. Based on the current system state $S = (G_{\text{sem}}, P_{\text{world}})$ and the Blockly rules formulated by the user, the system synthesizes an action command sequence A .
- Static Safety Validation:** We define a pre-execution verification function $V_{\text{safe}}(A) \rightarrow \{\text{True}, \text{False}\}$. This imposes a strict constraint: all target physical coordinates P_{world} within sequence A must be entirely contained within the camera's effective safe workspace W_{safe} . Commands are dispatched to the serial port only if $V_{\text{safe}}(A) = \text{True}$.

3.3. System Design Goals

Based on the formalized abstractions, the proposed framework must satisfy three fundamental design goals.

Goal 1: Semantic-Geometric Consistency Constraint. For any target object O_{target} specified in the visual logic $g(K_{\text{proc}})$, the system state must simultaneously satisfy semantic presence and geometric validity:

$$O_{\text{target}} \in G_{\text{sem}} \wedge \exists P_{\text{world}} (P_{\text{world}} = M_{\text{geo}}(M_{\text{det}}(I, O_{\text{target}}))) \quad (2)$$

Goal 2: Proactive Physical Safety Constraint. The generated action sequence A must be strictly bounded within the safe working space W_{safe} . Physical commands are dispatched if and only if the static verification $V_{\text{safe}}(A)$ holds:

$$\forall P_{\text{world}} \in A, \quad P_{\text{world}} \subseteq W_{\text{safe}} \iff V_{\text{safe}}(A) = \text{True} \quad (3)$$

Goal 3: Edge Deployability & Low-Code Efficiency. The framework imposes strict limits on computational resource $\text{Cost}(\cdot)$ for local edge devices (R_{edge}) and human orchestration time $\text{Time}(\cdot)$ relative to traditional direct coding (f):

$$\text{Cost}(M_{\text{vlm}}) + \text{Cost}(M_{\text{det}}) \leq R_{\text{edge}} \quad (4)$$

$$\text{Time}(g(K_{\text{proc}})) \ll \text{Time}(f(K_{\text{proc}}, S_{\text{code}})) \quad (5)$$

4. Methodology

This section details the methodology behind our low-code robotic control framework. The core challenge addressed by our system design (Fig. 2) is seamlessly translating high-level human intents into low-level, safety-verified robotic commands through a closed-loop pipeline. To make the data flow and system mechanisms concrete, the subsequent subsections are anchored by a continuous running example: the autonomous sorting of a red cube to a target location (200, 50). Through the lens of this specific task, we will unpack how the system processes visual block authoring, performs hybrid semantic-geometric perception, and guarantees safe physical execution.

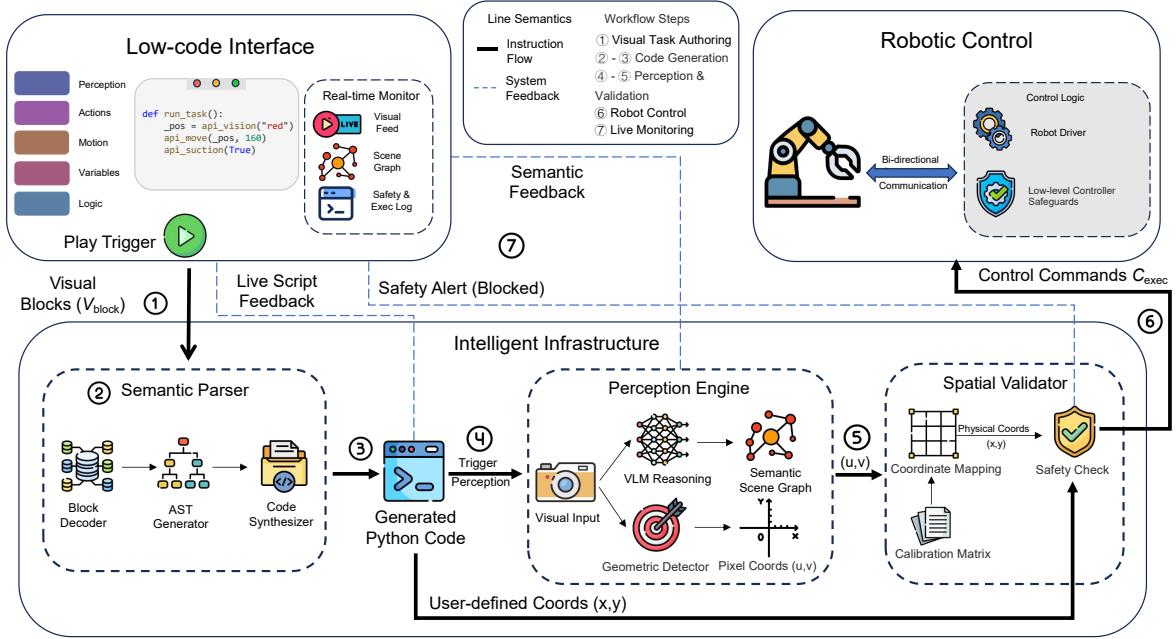


Figure 2: The overall architecture of the proposed low-code robotic control framework. The system comprises three core layers: the Low-code Interface, which includes visual task authoring and real-time monitoring; the Intelligent Infrastructure, which includes the semantic parser, perception engine, and spatial validator; and the Robotic Control, which includes robot drivers and low-level safeguards. Data flow steps ①–⑦ are detailed in Section 4.1.

4.1. System Overview

The proposed framework features a tripartite modular architecture tailored for non-expert users. The Low-Code Interface (shown on the top-left) captures user intents and provides real-time multi-modal feedback. Operating as the core computing engine, the Intelligent Infrastructure (spanning the bottom) handles semantic parsing, hybrid perception, and spatial validation. Finally, the Robotic Control module (on the top-right) is responsible for driving the physical hardware with low-level safeguards.

Data Flow of the Running Example: As illustrated in Fig. 2, the workflow follows seven distinct steps (①–⑦). The user's intention is captured via Visual Task Authoring (①) → processed by the Semantic Parser to generate Python code (②–③) → triggering the Perception Engine for dynamic target extraction (④) → validated by the Spatial Validator alongside user-defined coordinates (⑤) → dispatched as Executable Control Commands (C_{exec}) to the robot (⑥) → while continuously streaming Live Monitoring feedback back to the UI (⑦).

4.2. Low-Code Interface Design

The Low-Code Interface serves as the primary interaction gateway to bridge the gap between complex robotic operations and the domain knowledge of vocational workers. As illustrated in Fig. 3(a), the task orchestration component provides a library of semantic and atomic blocks categorized into five functional groups: Perception (dark blue), Actions (purple), Motion (brown), Variables (pink), and Logic & Loop (steel blue). This system provides a

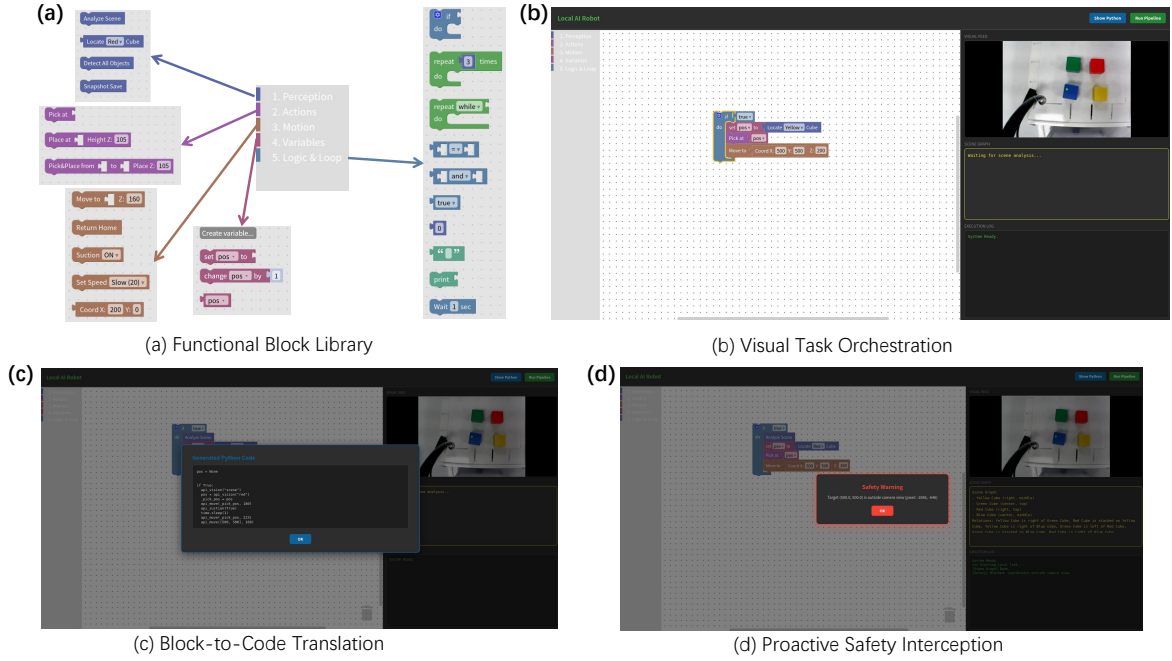


Figure 3: Detailed design of the Low-code Interface. (a) Functional block library categorized by the "Cognitive Scaffold" principle; (b) visual task orchestration on the workspace canvas (V_{block}); (c) automatic block-to-code translation for technical transparency (C_{exec}); (d) proactive safety warning triggered by an out-of-boundary coordinate input (500,500).

clear "Cognitive Scaffold," allowing users to focus on high-level process logic. For instance, in our running example (Fig. 3b), a worker can intuitively use a variable block to store the target coordinates obtained from an "api_vision" block and snap it into an "api_move" block. The workspace utilizes a block-to-code transparency mechanism (Fig. 3c) and visual syntax checking to ensure logical integrity. As Fraser (2015) highlighted, the physical constraints imposed by block shapes prevent the formation of syntactical invalid code.

To ensure the transparency of the autonomous perception modules and foster user trust, the interface incorporates a multi-modal monitoring system for real-time telemetry and feedback. The Visual Feed renders a live stream of the robot's workspace, while the Scene Graph dynamically visualizes the results of environmental parsing. For instance, in the red cube sorting task, the Scene Graph displays spatial relations such as "Red Cube is left of Green Cube" (as shown in Fig. 3d), providing immediate verification of the perception engine's output. This visual-semantic alignment allows the operator to verify whether the system's spatial understanding correctly reflects reality. In parallel, the Execution Log provides high-fidelity feedback by surfacing human-readable safety alerts. If a user accidentally inputs a restricted coordinate like (500, 500), the system intercepts the command and triggers a "Safety Warning" (Fig. 3d), stating "Target is outside camera view," enabling intuitive debugging without the need to inspect low-level error codes.

4.3. Intelligent Programming Infrastructure

The Intelligent Infrastructure serves as the system's computational core, responsible for translating visual logic into executable code and grounding high-level semantic intents into precise physical actions. This process is divided into three functional modules: the semantic parser, the perception engine, and the spatial validator.

4.3.1. Block-to-Code Generation

When the user triggers the execution, the Semantic Parser decodes the visual script (V_{block}) into an intermediate Abstract Syntax Tree (AST). The Code Synthesizer then performs a recursive traversal of the AST to map each visual node to a predefined Atomic API in the Python sandbox. In our red cube sorting example, the system identifies the "Find Object" and "Move To" blocks, synthesizing them into a coherent script: `_pos = api_vision("red")` followed by `api_move(_pos, 160)`. This generation mechanism ensures that complex industrial logic is reduced to a sequence of verifiable function calls, providing a stable foundation for the subsequent perception and execution phases.

4.3.2. Hybrid Semantic-Geometric Perception

To bridge the gap between high-level descriptions and low-level sensor data, the Perception Engine employs a dual-track mechanism that integrates semantic reasoning with geometric precision. The Semantic Layer utilizes a lightweight Vision-Language Model (VLM) (Xiao, Wu, Xu, Dai, Hu, Lu, Zeng, Liu and Yuan, 2024) to perform contextual reasoning. As demonstrated in recent research on visual instruction tuning, properly aligned VLMs can effectively interpret user instructions to generate a Semantic Scene Graph (Liu, Li, Wu and Lee, 2023). This graph identifies all objects and their relative positions, providing the necessary environmental context for the system.

Complementing this, the Geometric Layer employs a Geometric Detector (YOLOv8) (Jocher, Chaurasia and Qiu, 2023) to resolve the VLM's inherent lack of spatial precision. It extracts the precise pixel centroid (u, v) for the target class specified in the script. In the red cube sorting task, while the VLM confirms the presence and relational context of the cubes (e.g., "Red Cube is left of Green Cube"), the geometric detector provides the exact pixel coordinates (140, 37) for the target block, thereby enabling high-precision manipulation.

4.3.3. Spatial Mapping and Calibration

The final stage of the infrastructure involves the Spatial Validator, which establishes a bridge from the 2D image space to the 3D physical workspace. Adopting classical hand-eye calibration methodologies (Tsai and Lenz, 1989), the Coordinate Mapping module applies a calibrated 2D Affine Transformation Matrix (M) to the extracted pixel coordinates (u, v) to calculate the real-world world coordinates (x, y) . This transformation is governed by the equation:

$$P_{\text{world}} = M \cdot [u, v, 1]^T \quad (6)$$

In our example, the pixel position (140, 37) is mapped to the robot's base coordinate (228.0, 77.1). By integrating the perception engine's output with the calibration matrix, the system transforms abstract "pixel hits" into actionable physical waypoints, completing the grounding process necessary for robot motion.

4.4. Pre-execution Validation and Command Execution

4.4.1. Pre-execution Safety Check

Before any command is dispatched, the system intercepts the generated script. Utilizing the Safety Check module within the Spatial Validator (Fig. 2), and drawing upon classical robot collision-checking and industrial safety standards (Pan, Chitta and Manocha, 2012; Villani, Pini, Leali and Secchi, 2018), it extracts all target coordinates, including the dynamic pick point (228.1, 77.0) and the user-defined place point (200, 50). The module then applies an inverse transformation matrix from world coordinates to pixel coordinates to verify whether all targets fall within the camera's effective safe workspace.

4.4.2. Physical Execution

Once the safety check is passed, the validated coordinates are translated into low-level Control Commands (C_{exec}) and dispatched to the Robotic Control layer. The Robot Driver formats these coordinates into proprietary protocols, such as the command `DescartesPoint_228.1, 77.0, z`. These instructions are then flushed sequentially to the hardware through the communication interface.

During this phase, the system maintains bi-directional communication with the robot arm to ensure synchronized execution. While the Robot Driver manages the command flow, the Low-level Controller Safeguards continuously monitor hardware-specific constraints, such as joint torque limits and emergency stop triggers, providing a final layer of reactive safety. Furthermore, the robot's real-time operational data is transmitted back to the Intelligent Infrastructure to update the Real-time Monitor, physically closing the loop and confirming the successful completion of the task.

5. Experiments and Evaluation

In this section, we evaluate the proposed framework through two functional use cases and a comparative user study. The first two experiments validate the technical feasibility of the integrated perception-execution pipeline and the proactive safety mechanism.

5.1. Experimental Setup

The experiments were conducted using a 3-axis industrial robotic arm. A high-definition camera was mounted directly on the robotic wrist in an "Eye-in-Hand" configuration. This setup allows the robot to perform close-up inspections, though it requires the arm to move to a predefined Observation Pose to obtain a stable and calibrated top-down view of the entire workspace.

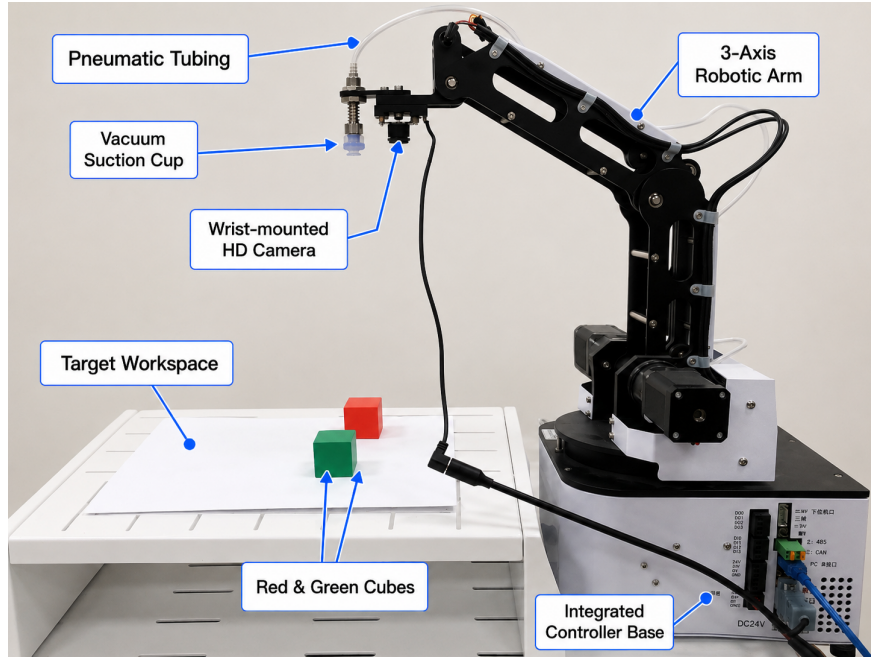


Figure 4: Experimental hardware platform. Component details of the 3-axis industrial robotic arm setup, featuring a wrist-mounted HD camera for "Eye-in-Hand" perception, a vacuum suction cup end-effector with pneumatic tubing, an integrated controller base, and a dedicated workspace containing the target red and green cubes.

Computational Environment: The intelligent infrastructure, including the Florence-2 VLM and YOLOv8s detector, was deployed on a local edge device to ensure low-latency processing.

Test Objects: Four industrial cubes of different colors (red, green, blue, and yellow) were used as targets.

5.2. Use Case I: Autonomous Pick-and-Place Task

The first use case validates the integration of the "Eye-in-Hand" perception with the execution pipeline. The task was defined as: *"Identify the environment, pick up the red cube, and stack it on top of the green cube."*

Perception Validation: To initiate the task, the robot first executed the `api_home()` command to reach the calibrated observation height (shown in Fig. 6a). From this fixed perspective, the Perception Engine invoked the VLM to generate a Semantic Scene Graph (Fig. 5), confirming that the cubes were within the reachable workspace and identifying their spatial relations. Simultaneously, the geometric detector (YOLOv8) extracted the centroid of the red cube at pixel coordinates (140, 37).

Execution Results: The system applied the 2D affine transformation to map the observation-pose pixels to physical world coordinates (228.1, 77.0). The robotic arm then descended from its observation height to the target (x, y) position to perform the grasp (Fig. 6b–d). The experimental results showed that even with the camera mounted on the moving arm, the system achieved consistent accuracy by utilizing a fixed-pose observation strategy, meeting the precision requirements for industrial sorting (Fig. 6e–f).

5.3. Use Case II: Pre-execution Safety Validation

This experiment evaluates the Proactive Physical Safety goal by simulating a scenario where a user inputs an erroneous or hazardous coordinate.

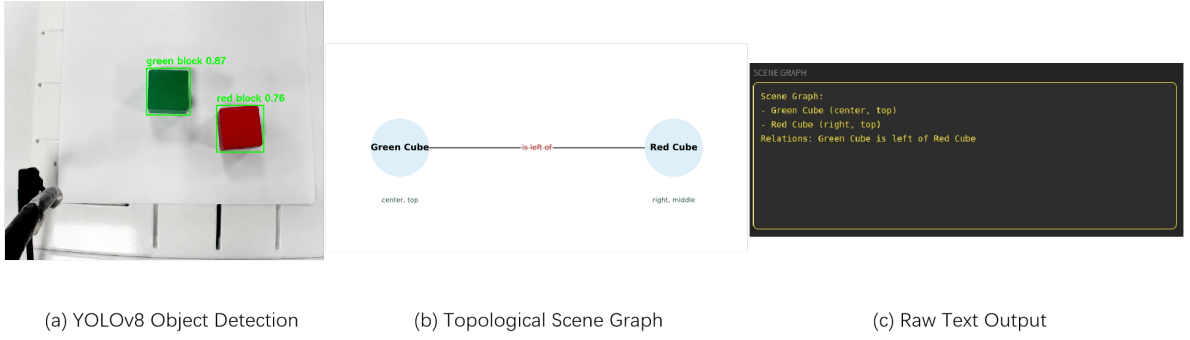


Figure 5: Visualization of the hybrid semantic-geometric perception process. (a) Real-time object detection and bounding box extraction using YOLOv8. (b) Topological representation of the semantic scene graph showing relative spatial relationships. (c) Raw text output of the scene graph generated by the lightweight Vision-Language Model (VLM).

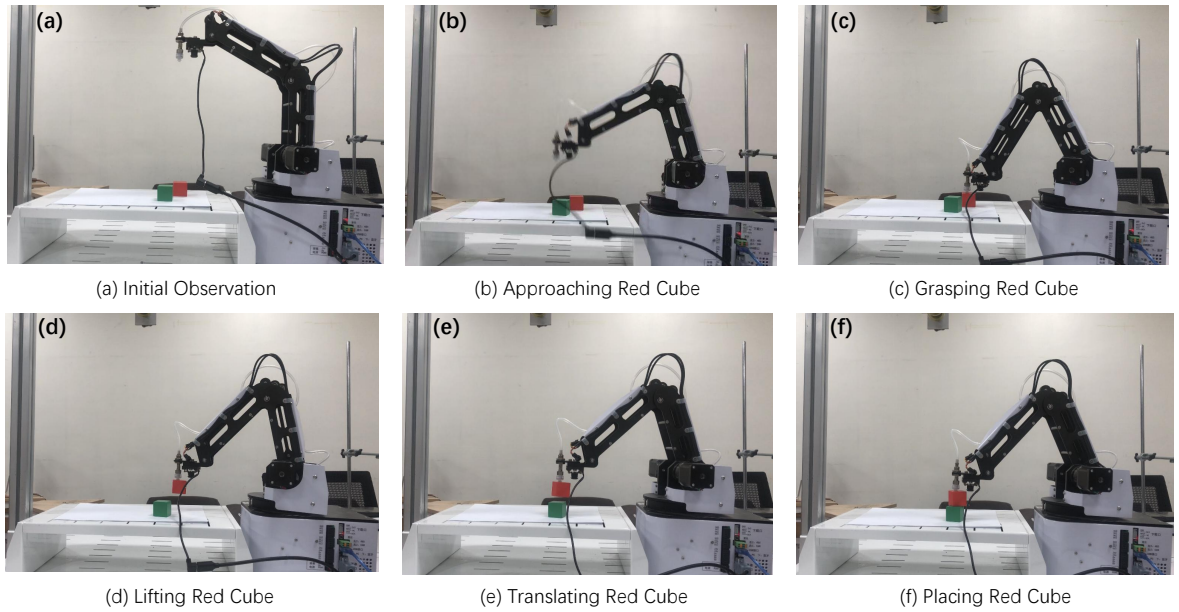


Figure 6: Sequential execution of the autonomous pick-and-place task. Step-by-step breakdown of the stacking operation: (a) initial observation pose, (b) approaching the target red cube, (c) grasping the red cube, (d) lifting the grasped object, (e) translating toward the stacking location, and (f) precisely placing the red cube on top of the green cube.

Experimental Scenario: A user attempted to move the robot to a target location (500, 500), which lies significantly outside the camera's effective field of view and the robot's safe operating radius.

Interception Workflow: Before the script was dispatched to the serial port, the Safety Check module intercepted the `api_move([500, 500], z)` command. By applying the inverse transformation matrix (World-to-Pixel), the system calculated that the target coordinate would reside at an unreachable pixel location far beyond the `CAM_MARGIN`.

Safety Defense Effect: The system immediately aborted the execution and triggered a Safety Warning on the monitoring panel (Fig. 7), stating: *"Target (500, 500) is outside camera view."* This result demonstrates that the framework effectively prevents potential hardware collisions and out-of-bound errors through static script analysis before any physical movement occurs, providing a high-tolerance "sandbox" for vocational training.

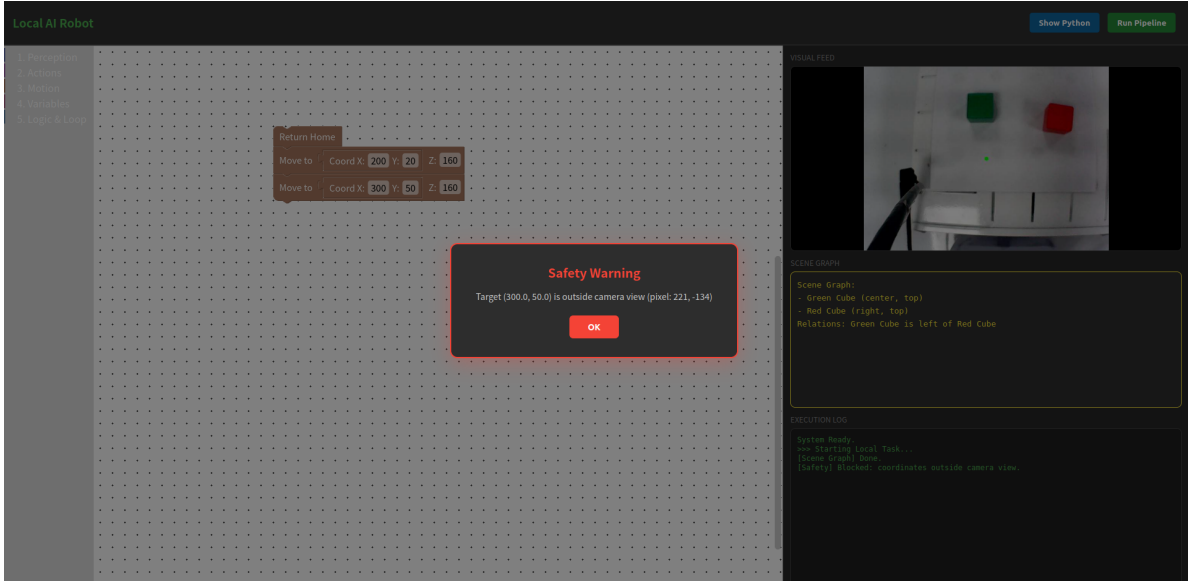


Figure 7: Pre-execution safety validation interface. Demonstration of the proactive collision avoidance mechanism, where an out-of-boundary coordinate input (e.g., target outside camera view) is successfully intercepted by the static safety checker, triggering a visual warning and halting execution before any physical movement.

5.4. Usability Analysis

To evaluate the effectiveness of the proposed low-code framework in lowering the barrier for robotic programming, we conducted a controlled user study. Building upon established methodologies in end-user robot programming evaluations (Weintrop, Afzal, Salac, Francis, Li, Shepherd and Franklin, 2018; Ajaykumar, Steele and Huang, 2021), the study aimed to compare the performance and subjective experience of users using traditional Python scripting versus the proposed Blockly-based interface.

5.4.1. Experimental Design

Participants and Recruitment: We recruited 8 volunteers (aged 18–25, 4 male and 4 female) via campus recruitment. The participant selection criteria focused on the "skill gap" defined in our problem statement: while they possessed moderate general programming knowledge (self-assessment in text-based languages including Python, C, and C++ averaging 3/5), they were total novices in the robotics domain. Crucially, 87.5% of the participants had never operated an industrial robot arm, ensuring that the study accurately reflects the learning curve for non-expert vocational workers who may have a basic logical foundation but lack hardware-specific expertise. All participants provided informed consent before the experiment.

Phase-Specific Participant Familiarization under Counterbalanced Design: To minimize the impact of software unfamiliarity on the experimental data, each of the two experimental phases began with an untimed, self-paced familiarization session. Since this study adopted a counterbalanced within-subject design where each participant executed tasks under both interfaces, this instructional onboarding was conducted sequentially. Specifically, before starting either the direct programming or the visual low-code tasks—depending on their assigned sequence—participants were required to study the corresponding "Operation Manual" and successfully run the provided sample operations under the researcher's guidance. The experimenters provided auxiliary explanations to resolve any syntactic or conceptual confusion. The official timed experimental tasks for each system commenced only after the participant explicitly confirmed they felt ready and fully understood the respective control logic, ensuring that individual differences in learning curves were neutralized before data collection.

Task Taxonomy and Cognitive Objectives: Four progressively challenging tasks were designed to evaluate the hierarchy of the control system's logic and the user's cognitive adaptation:

- **Task 1 (Basic Motion):** Evaluated sequential logic and basic block or API invocation, such as X-Y-Z coordinate entry and vacuum suction control.

- **Task 2 (Single-Object Perception):** Evaluated the user's ability to integrate the perception engine's output with motion commands, primarily utilizing the `api_vision` command.
- **Task 3 (Multi-Object Coordination):** Tested complex state management, requiring the user to orchestrate multiple "Pick-Home-Place" cycles to avoid camera occlusion.
- **Task 4 (Dynamic Conditional Logic):** Stood as the most complex task, requiring participants to implement conditional branching and iterative loops to selectively clear the workspace. This final task directly validates whether the system successfully maps the user's high-level process knowledge (K_{proc}) into executable commands without requiring formal textual coding skills.

Quantitative and Subjective Metrics: During the formal experiment, we utilized both automated performance logging and multi-layered subjective surveys to evaluate the framework:

- **Task Completion Time (TCT):** Measured using a stopwatch, starting from the moment the participant began writing code or arranging blocks until the robot successfully completed the physical action.
- **Error and Iteration Count:** Meticulously tracked by the researchers, who logged the frequency of failed executions, categorized into syntax errors (such as misspelled APIs or incorrect indentation in Python) and logic errors (such as incorrect coordinates or improper motion sequences). To preserve the integrity and objectivity of the evaluation, the researchers maintained a strict non-interventionist stance. A standardized hint was only provided and logged as an assistance count when a participant, unable to identify an error independently, explicitly requested help to proceed with the task.
- **Single Ease Question (SEQ):** Answered immediately after completing each specific task, participants provided a Perceived Ease of Use score on a 5-point Likert scale (1 = Hardest, 5 = Easiest) as defined by Sauro and Dumas (2009).
- **System Usability Scale (SUS):** Completed upon finishing all tasks within a specific programming paradigm, either Python or Blockly, using the standard 10-item questionnaire established by Brooke (1996) to evaluate the overall interface usability.
- **Semi-structured Interview:** Conducted at the end of the session, capturing qualitative feedback regarding participants' perceived physical control, programming pain points, and overall interface preference.

5.4.2. Quantitative Evaluation

Task Completion Time Analysis: The quantitative results from the user study demonstrate a profound efficiency advantage for the Blockly-based low-code framework across all experimental stages, as illustrated in the Task Completion Time Comparison (Fig. 8).

For Task 1 involving basic motion control, the Python group required an average of 326.8 seconds, whereas the Blockly group completed the same sequence in just 126.9 seconds, representing a 61.2% reduction in time cost. This efficiency gap widened significantly in Task 3 involving multi-object coordination, where the Python group spent an average of 540.6 seconds due to the cognitive overhead of manually scripting consecutive coordinates and intermediate homing sequences. In contrast, the Blockly group executed the task in 148.0 seconds, achieving a 72.6% time-saving. Even in Task 4, the most complex logic-testing environment, the Blockly group required only 435.3 seconds compared to the Python group's 960.5 seconds, saving 54.7% of the total completion time. These findings indicate that the low-code abstraction significantly accelerates the programming cycle for non-experts.

Error Rate and Debugging Iterations: The disparity in programming error rates underscores the low-code interface's effectiveness in mitigating operational friction, as illustrated in the Error Rate Comparison (Fig. 9). Throughout the experiment, the Python group consistently experienced high error rates across all tasks, peaking at an average of 3.25 errors per participant during the complex logical challenge of Task 4. Our qualitative analysis of the research logs reveals that these errors consistently fell into three distinct categories.

The first category was *syntactic and API misuse*, where participants routinely omitted parentheses in parameterless getter and actuator functions (such as executing `suction_on` or `get_lift_z` as raw variables) and struggled with Python's dictionary traversal syntax, resulting in typos like `doords` or incomplete `.items()` calls. The second was *workflow execution omissions*, primarily represented by participants forgetting to save their scripts before execution,

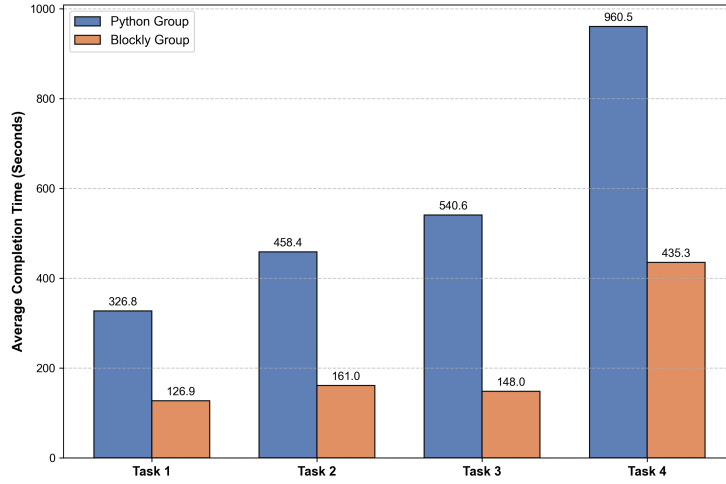


Figure 8: Average task completion time comparison. This bar chart illustrates the average time (in seconds) required by participants to complete four progressively challenging tasks. The Blockly-based group demonstrates significantly higher efficiency, with completion times consistently lower than the Python-based group across all tasks, most notably in Task 4, where the time-saving reaches 54.7%.

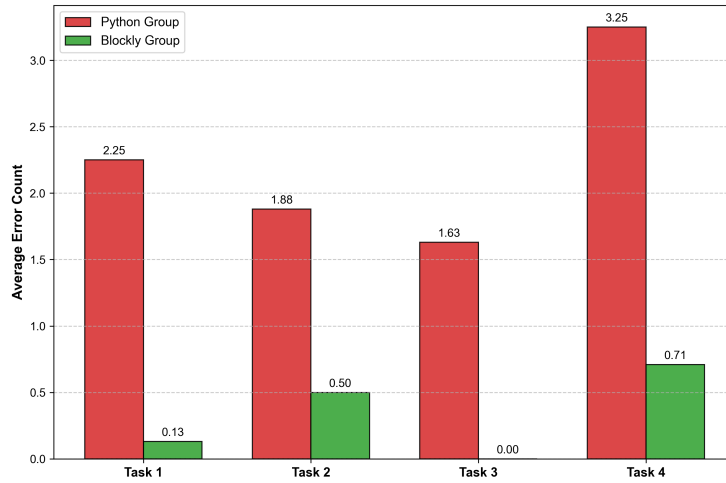


Figure 9: Average error count per task. This figure compares the average number of errors committed by participants in the Python and Blockly groups. The Python group exhibits a high frequency of syntax and physical control errors, particularly in Task 4. Conversely, the Blockly interface's visual grammar checking and structural constraints effectively eliminate most syntax-related and out-of-boundary errors, maintaining an average error rate of less than 1.0 across all tasks.

leading to silent failures. The third category involved *physical control and sequencing errors*, which directly threatened hardware safety. These included omitting the intermediate lift step (`get_lift_z()`) before lateral translation—causing the robotic arm to scrape against the tabletop—and inputting out-of-boundary coordinates like (200, −550). Most notably, participants often forgot to return the arm to the home position after a pick, which physically blocked the wrist-mounted camera's field of view and prevented the perception engine from scanning subsequent targets.

Conversely, the Blockly group maintained a remarkably low average error rate, peaking at just 0.71 in Task 4. The structural validation of the visual workspace successfully minimized physical hazards. While a few participants in the Blockly group made minor parameter errors by misconfiguring the vertical height (such as U02 and U05 in Task 1, and U04 in Task 2 due to setting the Z-axis too low), leading to a few isolated instances of minor tabletop collisions, these errors were easily corrected. Furthermore, although some participants did attempt to input out-of-boundary physical

coordinates, these hazardous movements were completely prevented from executing on the hardware, as they were successfully intercepted and blocked by the system's Pre-execution Safety Check, validating the framework's ability to enforce operational safety at the logic-authoring level.

5.4.3. Assistance Rate and Usability Implications

The frequency of active assistance requests serves as a critical proxy for evaluating the deep cognitive barriers and logical blind spots inherent in each paradigm, with the quantitative summary of these interventions compiled in Table 1.

Table 1

Comparison of Average Assistance/Hint Requests per Task (Mean $\pm$ SD)

Programming Paradigm	Task 1	Task 2	Task 3	Task 4
Python Group	0.75 ± 0.80	0.13 ± 0.35	0.25 ± 0.71	0.88 ± 1.13
Blockly Group	0.00 ± 0.00	0.13 ± 0.35	0.00 ± 0.00	0.38 ± 0.74
Reduction Rate	-100.0%	0.0%	-100.0%	-57.1%

As indicated in the data, the Blockly group achieved a flawless zero-assistance rate (0.00 ± 0.00) in both Task 1 and Task 3, representing a 100% reduction in intervention compared to the Python group's 0.75 ± 0.80 and 0.25 ± 0.71 respectively. This complete independence in the visual group stands in stark contrast to the text-based group, where participants frequently reached unresolvable operational deadlocks and explicitly requested experimenter intervention to proceed. The primary driver for these interventions was that many of the errors analyzed in Section 5.4.2 were entirely invisible to novice programmers, resulting in high cognitive anxiety. For instance, when Python participants forgot to save their text scripts, the local execution terminal provided no visual cues, leaving them completely bewildered as to why their physical inputs yielded no robotic response.

Similarly, omitting parentheses on getter functions (e.g., executing `get_lift_z` as a raw variable) did not trigger immediate syntax crashes in the Python sandbox, but silently returned a function pointer rather than the expected float coordinate, leading to obscure type errors downstream that novices lacked the debugging intuition to resolve. Furthermore, when participants forgot to return the robotic arm to the observation height, the wrist-mounted camera's field of view was physically blocked by the arm itself, creating an occlusion-induced perception deadlock that completely stalled their progress. In contrast, the Blockly group required substantially less assistance, maintaining an average request rate of just 0.38 ± 0.74 even in the highly complex logic challenge of Task 4, achieving a 57.1% reduction compared to Python's 0.88 ± 1.13 . By offering auto-compiled visual blocks and enforcing structural constraints, the visual framework neutralized syntactic, workflow, and occlusion-induced blind spots, proving that the low-code interface successfully flattens the learning curve and transforms stressful, syntax-constrained debugging into an intuitive, process-level task.

5.4.4. Subjective Usability Evaluation

The perceived cognitive load across the four progressive tasks was mapped using the post-task SEQ scores, as illustrated in Fig. 10.

While both programming paradigms started with relatively high ease-of-use scores in the basic movement task (4.38 for Python and 4.75 for Blockly), their trajectories diverged sharply as logical complexity scaled up. In the Python group, participants experienced a steep cognitive cliff, with their perceived ease of use plummeting to a frustrating 2.50 in Task 4, reflecting the high mental workload imposed by syntax debugging and loop structures. In contrast, the Blockly group demonstrated remarkable cognitive resilience, with ease-of-use scores only slightly slipping to 3.86 in Task 4. This minimal decline proves that the visual low-code framework successfully acts as an effective cognitive scaffold, keeping the task within the user's comfort zone even during complex logic orchestration.

The System Usability Scale (SUS) scores further validate the overall system acceptability and usability, as shown in Fig. 11.

The traditional Python scripting interface achieved an average SUS score of only 50.6, falling significantly below the widely accepted industry usability benchmark of 68, which categorizes its usability as marginal or poor. This low score highlights the high entry barrier of text-based languages for front-line operators. Conversely, the proposed Blockly-based low-code system achieved an average SUS score of 77.2, placing it well into the good-to-excellent

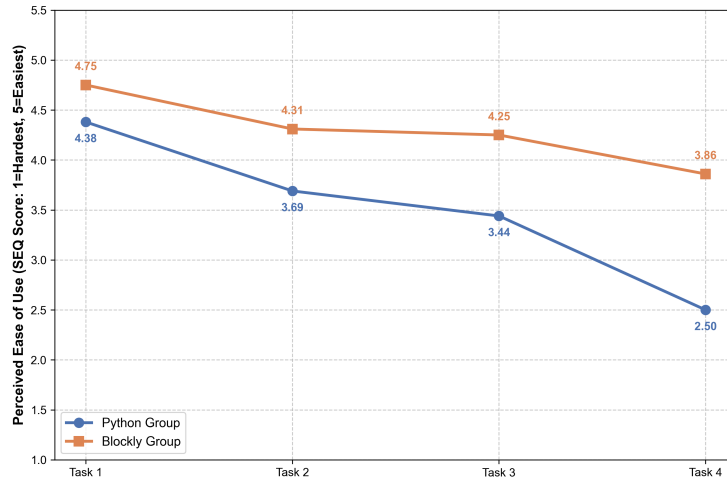


Figure 10: Perceived ease of use (SEQ) across experimental tasks. This line chart maps the cognitive difficulty experienced by participants on a 5-point Likert scale (1=Hardest, 5=Easiest). While both groups reported high ease-of-use in Task 1, the Python group's perceived difficulty increases sharply as logic complexity rises, reflecting a steep cognitive barrier. The Blockly group, however, exhibits remarkable cognitive resilience, maintaining a stable user experience throughout the entire session.

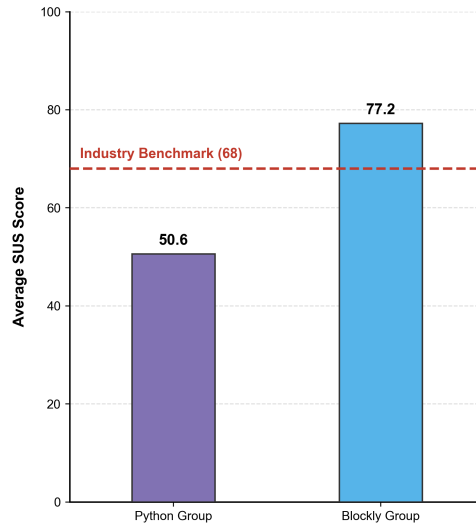


Figure 11: Average System Usability Scale (SUS) scores. The comparative analysis shows that the proposed Blockly-based low-code system (77.2) significantly outperforms the Python-based control paradigm (50.6). The Blockly system scores well above the industry usability benchmark of 68, confirming its effectiveness in facilitating non-expert industrial robot programming.

usability range. This high rating confirms that the visual framework is highly intuitive and ready for practical vocational training deployment.

Qualitative feedback from the semi-structured interviews corroborated the quantitative SUS scores, revealing distinct psychological experiences between the two programming paradigms. As summarized in Table 2, the Python group's negative feedback was dominated by themes of *syntactic complexity*, *logical fragility*, and *low operational efficiency*, with participants frequently citing "strict formatting" and the high cognitive cost of syntax debugging as primary deterrents.

Table 2

Thematic Analysis of Negative Feedback (Python Group)

Core Pain Points (Themes)	Frequency	Representative Quotes
Syntactic & Formatting Constraints	4	"Too many syntax rules", "Strict formatting", "Frequent typos"
Logical Complexity & Error Sensitivity	2	"Prone to errors", "High entry barrier", "Complex logic"
Low Input Efficiency	1	"Tedious manual code repetition"

Conversely, as shown in Table 3, the Blockly group's positive feedback centered on *intuitive logical flow* and *high operational efficiency*, with participants highlighting the ease of getting started and the elimination of manual coding errors as the system's key highlights.

Table 3

Thematic Analysis of Positive Feedback (Blockly Group)

Core Highlights (Themes)	Frequency	Representative Quotes
Intuitive Logical Flow	5	"Easy to learn", "Simple/Clear logic", "Intuitive"
High Operational Efficiency	3	"Reduced workload", "Convenient", "No manual coding"

While a significant majority of participants expressed a strong preference for the Blockly interface, one participant (U08) noted a preference for Python due to its superior capacity for fine-grained control and precise parameter adjustment. This divergence highlights a classic human-computer interaction trade-off between the cognitive accessibility of visual abstractions and the expressive flexibility of text-based environments. Our system attempts to mediate this trade-off through the "Show Python" feature, which functions as a cognitive bridge by providing real-time transparency into the underlying code generated from the visual logic. This aligns with prior human-computer interaction studies showing that dual-representation and hybrid environments significantly flatten the learning curve for novices (Weintrop and Wilensky, 2017), suggesting that the future of low-code robotic platforms lies in developing dual-representation interfaces that support a seamless transition from intuitive visual orchestration to professional-grade code control as the user's expertise scales (Wang et al., 2025).

5.4.5. Summary of Usability Findings

The experimental results collectively confirm the framework's efficacy in addressing the identified skill gap among vocational workers. The comparative study reveals that the proposed low-code environment significantly outperforms traditional text-based scripting in both objective performance and subjective user experience.

Specifically, our analysis highlights three key usability advancements. First, the **syntax-free interface** effectively eliminates common programming errors, such as misspellings and indentation failures, which were found to be the primary cause of frustration for novice users in the Python group. Second, the **visual scaffolding** provided by the block-based interface dramatically flattens the learning curve; while Python participants faced a "cognitive cliff" when encountering complex logic, Blockly participants maintained high performance and stability. Third, the **integrated safety and monitoring mechanisms** provided a proactive defense against physical hardware risks, such as tabletop collisions and out-of-boundary movements, which occurred unchecked in the textual environment.

In summary, the transition from code-level debugging to process-level orchestration demonstrates the high value of low-code platforms for industrial training. By shielding users from low-level syntax while preserving the rigors of industrial logic, our system empowers non-expert workers to maximize their potential in automated production environments. This validation confirms that the framework successfully achieves its design goals of semantic-geometric consistency, proactive safety, and edge-deployable efficiency.

6. Conclusion

This paper has presented the design and development of a comprehensive web-based, low-code robotic arm control framework, specifically targeting the critical need to expedite the training of vocational workers in industrial environments. By integrating a visual Blockly interface with a hybrid semantic-geometric perception pipeline, we

have successfully addressed the "skill gap" that hinders non-expert operators from leveraging modern industrial robotics. Our system architecture, which combines lightweight Vision-Language Models (VLM) for semantic scene understanding with geometric detectors for precise localization, provides a robust, edge-deployable solution for task-level programming. Furthermore, the implementation of a proactive, pre-execution static safety check ensures operational integrity, mitigating the risks associated with manual script generation in unconstrained text-based environments.

The experimental evaluation confirms that the proposed framework achieves its design objectives of semantic-geometric consistency, proactive safety, and high operational efficiency. Our user study demonstrates a statistically significant improvement in task completion time and a profound reduction in programming errors compared to traditional Python scripting. By abstracting lower-level syntax into semantic visual blocks, our system successfully transforms robotic programming from a high-barrier, syntax-constrained debugging task into an intuitive, process-level orchestration. This shift not only lowers the entry barrier for existing skilled field workers but also maximizes their potential by allowing them to focus on the process knowledge they already possess, rather than the complexities of text-based code.

While the proposed framework establishes a robust foundation for low-code robotic control, it opens several significant avenues for future research. First, we intend to integrate high-fidelity Digital Twin technologies to narrow the "reality gap" between simulation and physical deployment. By establishing a real-time, bi-directional synchronized virtual replica, we aim to enhance offline programming fidelity, enabling operators to commission complex production sequences with higher confidence without disrupting active industrial workflows.

Second, we plan to incorporate Large Language Model (LLM) agents to facilitate conversational programming. By enabling the system to perform "Intent-to-Logic" synthesis, we aim to offload complex procedural reasoning tasks from operators. This would allow workers to describe desired tasks in natural language, which the LLM would then decompose into optimized Blockly sequences. This evolution would further minimize the cognitive load during large-scale production line reconfigurations. Finally, we will extend the current architecture to support multi-arm collaborative orchestration. Scaling the framework to handle heterogeneous robotic systems will enable the system to address the requirements of large-scale industrial assembly processes, where coordinated multi-robot interaction and synchronization are essential.

Open Science and Artifact Availability

To support scientific replication and transparency, the complete source code of the low-code interface, the semantic parser pipeline, the raw user study dataset, and experimental demonstration videos are publicly available on our companion repository: <https://github.com/username/repository>.

A. Participant Demographic Profile

To ensure the reliability of the usability evaluation, eight participants (U01–U08, aged 18–25, 4 male and 4 female) were recruited. Table 4 compiles their demographic background and self-evaluated programming experience (scored on a 1–5 scale, where 1 = Novice, 5 = Expert). Crucially, 87.5% of the participants had never operated a physical industrial robotic arm before, representing the targeted non-expert workspace operators.

Table 4
Demographic and programming background of the participants

Participant ID	Gender	Age	Python Experience	Blockly Experience	Prior Robotic Arm Experience
U01	Female	18–25	3	1	Heard of it only
U02	Male	18–25	3	1	Never operated
U03	Male	18–25	1	1	Never operated
U04	Female	18–25	4	2	Heard of it only
U05	Female	18–25	3	1	Heard of it only
U06	Female	18–25	3	3	Never operated
U07	Male	18–25	4	1	Minor operating experience
U08	Male	18–25	3	3	Never operated

Table 5

Mapping of Core Visual Blocks to Underlying Python APIs

Category	Visual Block	Executable Python Command	Functional Description
Perception	Analyze Scene Graph	<code>detect_all()</code>	Scans the workspace and outputs semantic labels.
Perception	Locate [Color] Cube	<code>locate("Color")</code>	Returns [x, y] pixel coordinates of target.
Actions	Pick at [Position]	Encapsulated sequence: <code>move()</code> , <code>suction_on()</code>	Handles alignment, approach, grasping, and lifting.
Actions	Place at [Position]	Encapsulated sequence: <code>move()</code> , <code>suction_off()</code>	Handles translation, lowering, releasing, and safety reset.
Motion	Return Home	<code>home()</code>	Returns the robotic arm to the origin.
Motion	Move to [X/Y/Z]	<code>move(x, y, z)</code>	Manual linear movement to specific coordinates.
Motion	Wait [n] sec	<code>wait(n)</code>	Introduces a workflow pause.

B. Experimental Tasks Specifications

The usability test consisted of four progressively challenging robotic manipulation tasks:

- **Task 1 (Basic Motion):** Verify basic motion control. Control the robotic arm from the origin position to coordinates (200, 30, 200), activate the vacuum suction cup, wait for 2 seconds, deactivate the suction, and return home.
- **Task 2 (Single-Object Perception & Pick-and-Place):** Automate the visual detection and grasping of a red cube, place it at coordinates (200, 30), and return the arm to the origin.
- **Task 3 (Multi-Object Coordination):** Implement a continuous sorting sequence: pick up and place the red cube at (200, 30), return to home position to avoid camera occlusion, and then pick and place the blue cube at (170, 30).
- **Task 4 (Dynamic Conditional Logic):** Scan the workspace and conditionally pick and discard all non-red cubes (e.g., blue, green, yellow) into a trash bin at (200, -50), while keeping the red cube untouched. Return to home after each discard.

C. Visual Block-to-API Mapping Reference

Table 5 details the core visual intermediate representations (V_{block}) defined in our framework and how they map to the low-level executable command sequences (C_{exec}) via the automated compiler.

D. User Study Instruments and Questionnaires

Three evaluation instruments were utilized to gather quantitative and qualitative insights:

- **Single Ease Question (SEQ):** Administered immediately after each task. Participants answered: "Overall, how easy or difficult was it to complete this task? (1 = Very Difficult, 5 = Very Easy)".
- **System Usability Scale (SUS):** Administered after completing all tasks under each programming paradigm (Python and Blockly). We utilized the standard 10-item questionnaire scored on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree).
- **Post-Study Semi-Structured Interview:** Captured qualitative feedback. Key prompts focused on: (1) debugging bottlenecks, (2) the confidence brought by visual syntax checking, and (3) trade-offs between visual block modularity and text-based granularity.

E. Participant-Level Detailed Performance Data

To provide maximum scientific transparency and facilitate meta-analysis, Table 6 compiles the comprehensive, participant-level raw performance metrics (Task Completion Time in seconds and Error Count) for all eight participants (U01–U08) across both the traditional Python and the proposed Blockly paradigms.

Table 6
Individual Performance Metrics Across All Four Tasks

Participant ID	System	T1 Time (Err)	T2 Time (Err)	T3 Time (Err)	T4 Time (Err)	Total Help
U01	Python	225s (3)	988s (3)	406s (0)	645s (0)	2
	Blockly	131s (0)	93s (0)	155s (0)	692s (2)	2
U02	Python	296s (1)	503s (2)	486s (0)	1339s (4)	2
	Blockly	161s (0)	142s (0)	235s (0)	155s (0)	0
U03	Python	609s (4)	434s (3)	626s (3)	1258s (6)	3
	Blockly	93s (0)	356s (3)	171s (0)	374s (0)	1
U04	Python	538s (5)	313s (0)	1059s (5)	1424s (8)	6
	Blockly	101s (0)	220s (1)	149s (0)	1402s (3)	1
U05	Python	285s (1)	261s (0)	450s (1)	600s (0)	1
	Blockly	148s (1)	111s (0)	126s (0)	162s (0)	1
U06	Python	102s (0)	524s (4)	331s (0)	483s (1)	0
	Blockly	79s (0)	123s (0)	106s (0)	247s (0)	0
U07	Python	305s (2)	357s (3)	527s (3)	657s (0)	1
	Blockly	137s (0)	92s (0)	74s (0)	157s (0)	0
U08	Python	254s (2)	287s (0)	440s (2)	1278s (7)	1
	Blockly	165s (0)	151s (0)	168s (0)	293s (0)	1

F. Individual SUS Scores and System Preference

Table 7 presents the detailed 10-item System Usability Scale (SUS) scores (Q_1 – Q_{10}) for each participant under both control interfaces, along with their calculated final SUS scores and overall system preferences.

References

- Ajaykumar, G., Steele, M., Huang, C.M., 2021. A survey on end-user robot programming. *ACM Comput. Surv.* 54. URL: <https://doi.org/10.1145/3466819>, doi:10.1145/3466819.
- Brooke, J., 1996. SUS: A 'Quick and Dirty' Usability Scale, in: *Usability Evaluation In Industry*. CRC Press. Num Pages: 6.
- Erdei, T.I., Krakó, R., Husi, G., 2022. Design of a Digital Twin Training Centre for an Industrial Robot Arm. *Applied Sciences* 12. URL: <https://www.mdpi.com/2076-3417/12/17/8862>, doi:10.3390/app12178862.
- Fischer, K., Kirstein, F., Jensen, L.C., Krüger, N., Kukliński, K., aus der Wieschen, M.V., Savarimuthu, T.R., 2016. A comparison of types of robot control for programming by demonstration, in: *2016 11th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pp. 213–220. doi:10.1109/HRI.2016.7451754.
- Fraser, N., 2015. Ten things we've learned from blockly, in: *2015 IEEE Blocks and Beyond Workshop (Blocks and Beyond)*, pp. 49–50. doi:10.1109/BLOCKS.2015.7369000.
- Jocher, G., Chaurasia, A., Qiu, J., 2023. Ultralytics YOLOv8. GitHub repository. URL: <https://github.com/ultralytics/ultralytics>. accessed: 2026-05-26.
- Kamel, K., Kamel, E., 2019. Process control ladder logic trouble shooting techniques fundamentals. *IRO Journal on Sustainable Wireless Systems* 1, 206–214. URL: <https://irojournals.com/iros/ws/article/view/1091>, doi:10.36548/jsws.2019.4.001.
- Kodosky, J., 2020. Labview. *Proc. ACM Program. Lang.* 4. URL: <https://doi.org/10.1145/3386328>, doi:10.1145/3386328.
- Kumar, L., Jetley, R., Sureka, A., 2016. Source code metrics for programmable logic controller (plc) ladder diagram (ld) visual programming language, in: *Proceedings of the 7th International Workshop on Emerging Trends in Software Metrics*, Association for Computing Machinery, New York, NY, USA, p. 15–21. URL: <https://doi.org/10.1145/2897695.2897699>, doi:10.1145/2897695.2897699.
- Liu, H., Li, C., Wu, Q., Lee, Y.J., 2023. Visual instruction tuning. URL: <https://arxiv.org/abs/2304.08485>, arXiv:2304.08485.

Table 7
Individual Raw SUS Question Scores and Subjective Preferences

Participant ID	System	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	SUS Score	Preference
U01	Python	5	3	4	3	4	2	4	3	3	2	67.5	Blockly
	Blockly	4	3	3	4	4	2	3	2	3	3	57.5	
U02	Python	1	5	2	4	2	3	4	5	3	4	27.5	Blockly
	Blockly	3	3	4	2	3	3	4	2	4	2	65.0	
U03	Python	3	2	4	2	3	3	4	2	3	2	65.0	Blockly
	Blockly	5	1	5	1	4	3	5	1	5	1	92.5	
U04	Python	4	3	3	4	4	2	2	3	4	4	50.0	Blockly
	Blockly	4	2	3	3	3	1	4	2	4	2	70.0	
U05	Python	4	2	4	4	4	1	3	2	4	2	70.0	Blockly
	Blockly	4	1	5	3	4	1	4	1	5	1	87.5	
U06	Python	3	4	2	4	4	2	2	4	2	3	40.0	Blockly
	Blockly	4	1	5	1	4	1	5	1	4	2	90.0	
U07	Python	2	4	2	2	3	2	1	4	3	3	40.0	Blockly
	Blockly	3	3	2	2	4	2	4	2	4	1	70.0	
U08	Python	3	4	2	3	5	3	2	3	3	4	45.0	Python
	Blockly	4	1	5	4	5	1	4	1	5	1	85.0	

- Mayr-Dorn, C., Winterer, M., Salomon, C., Hohensinger, D., Ramler, R., 2021. Considerations for using block-based languages for industrial robot programming - a case study, in: 2021 IEEE/ACM 3rd International Workshop on Robotics Software Engineering (RoSE), pp. 5–12. doi:10.1109/RoSE52553.2021.00008.
- Pan, J., Chitta, S., Manocha, D., 2012. Fcl: A general purpose library for collision and proximity queries, in: 2012 IEEE International Conference on Robotics and Automation, pp. 3859–3866. doi:10.1109/ICRA.2012.6225337.
- Sauro, J., Dumas, J.S., 2009. Comparison of three one-question, post-task usability questionnaires, in: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, New York, NY, USA. p. 1599–1608. URL: <https://doi.org/10.1145/1518701.1518946>, doi:10.1145/1518701.1518946.
- Sehr, M.A., Lohstroh, M., Weber, M., Ugalde, I., Witte, M., Neidig, J., Hoeme, S., Niknami, M., Lee, E.A., 2021. Programmable logic controllers in the context of industry 4.0. IEEE Transactions on Industrial Informatics 17, 3523–3533. doi:10.1109/TII.2020.3007764.
- Steinmetz, F., Nitsch, V., Stulp, F., 2019. Intuitive task-level programming by demonstration through semantic skill recognition. IEEE Robotics and Automation Letters 4, 3742–3749. doi:10.1109/LRA.2019.2928782.
- Tsai, R., Lenz, R., 1989. A new technique for fully autonomous and efficient 3d robotics hand/eye calibration. IEEE Transactions on Robotics and Automation 5, 345–358. doi:10.1109/70.34770.
- Villani, V., Pini, F., Leali, F., Secchi, C., 2018. Survey on human–robot collaboration in industrial settings: Safety, intuitive interfaces and applications. Mechatronics 55, 248–266. URL: <https://www.sciencedirect.com/science/article/pii/S0957415818300321>, doi:10.1016/j.mechatronics.2018.02.009.
- Wang, J., Shi, E., Hu, H., Ma, C., Liu, Y., Wang, X., Yao, Y., Liu, X., Ge, B., Zhang, S., 2025. Large language models for robotics: Opportunities, challenges, and perspectives. Journal of Automation and Intelligence 4, 52–64. URL: <https://www.sciencedirect.com/science/article/pii/S2949855424000613>, doi:10.1016/j.jai.2024.12.003.
- Weintrop, D., 2019. Block-based programming in computer science education. Commun. ACM 62, 22–25. URL: <https://doi.org/10.1145/3341221>, doi:10.1145/3341221.
- Weintrop, D., Afzal, A., Salac, J., Francis, P., Li, B., Shepherd, D.C., Franklin, D., 2018. Evaluating coblox: A comparative study of robotics programming environments for adult novices, in: Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, New York, NY, USA. p. 1–12. URL: <https://doi.org/10.1145/3173574.3173940>, doi:10.1145/3173574.3173940.
- Weintrop, D., Shepherd, D.C., Francis, P., Franklin, D., 2017. Blockly goes to work: Block-based programming for industrial robots, in: 2017 IEEE Blocks and Beyond Workshop (B&B), pp. 29–36. doi:10.1109/BLOCKS.2017.8120406.
- Weintrop, D., Wilensky, U., 2017. Between a block and a typeface: Designing and evaluating hybrid programming environments, in: Proceedings of the 2017 Conference on Interaction Design and Children, Association for Computing Machinery, New York, NY, USA. p. 183–192. URL: <https://doi.org/10.1145/3078072.3079715>, doi:10.1145/3078072.3079715.
- World Economic Forum, . The Future of Jobs Report 2025. URL: <https://www.weforum.org/publications/the-future-of-jobs-report-2025/>.

- Xiao, B., Wu, H., Xu, W., Dai, X., Hu, H., Lu, Y., Zeng, M., Liu, C., Yuan, L., 2024. Florence-2: Advancing a unified representation for a variety of vision tasks, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4818–4829.
- Zeng, F., Gan, W., Huai, Z., Sun, L., Chen, H., Wang, Y., Liu, N., Yu, P.S., 2025. Large language models for robotics: A survey. URL: <https://arxiv.org/abs/2311.07226>, arXiv:2311.07226.